CareerPilot AI

Toluwani Aderibigbe
3050667

Submitted in partial fulfilment for the degree of
B.Sc. in Computing Science

Griffith College Dublin
June 2026

Under the supervision of
Ellen Hickey

Disclaimer

I hereby certify that this material, which I now submit for assessment on the programme of study leading to the Degree of Bachelor of Science in Computing at Griffith College Dublin, is entirely my own work and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

Signed: ___Ade_____ **Date:** _3/9/2026_____

Acknowledgements

Here you can thank your family, colleagues, etc.

Table of Contents

Abstract

The project demonstrate the design and development of CareerPilot AI, an intelligent web-based career development platform developed using React, FastAPI, and Artificial Intelligence. The platform enables users to create an account, securely log in, upload a PDF CV and compare it with a selected job description. The system extracts information from the uploaded CV, identifies the matching skills and missing skills

Chapter 1: Introduction

1.1 Project Context and Problem Definition

CareerPilot AI AI is a web-based career supporting platform developed to help students and graduates prepared for job applications. which was developed using React with vite for the frontend and python with FastAPI for the backend. The platform includes features such as user registration and login, CV upload, CV analysis, job description comparison, career recommendations, cover letter generation, interview practice and can also generate cover letter. The system also uses AI to help us for interview questions preparation and cover letter generation to improve their job applications.

The main goal of the project is to allow user upload their cv, and enter their job description. and also combine different career preparation tools into one application and ensure they work smoothly together. many people use different websites to analyse their CV, generate cover letters, prepare for interviews and identify missing skills. Switching between multiple platforms which takes time and frustrating. so CareerPilot AI solves this problem by providing these services in a single system to make it very easy and fast for people who are planning to prepare their cv to get job.

Another important part of the project is the CV analysis feature. The system allows users to upload a PDF CV and compare it with a job description. The application extracts the text from the CV, identifies important skills and compares them with the skills required for the selected job. It then shows the matched skills, missing skills, a match score and a suitable career recommendations to help users understand how well they are qualified for the position they are trying to apply for.

The project also includes AI-powered features such as cover letter generation and interview practice. These features provide content based on the user's CV and the selected job role, helping users prepare more effectively for job applications and interviews. In addition, the platform provides a simple and responsive user interface that allows users to navigate easily between different features. The aim of the project is to allow user register, login so when they login they will be able to upload their cv

and enter their job description. The main problem faced during development was making all the parts of the application work together, The React frontend, FastAPI backend, SQLite database, ESCO data and Google Gemini API all need to communicate in the right way so, that information can be processed and the correct results can be displayed to the user.

1.2 Project Goals and Objectives

The main goal of CareerPilot AI is to develop a web-based career assisting platform that helps users understand their skills and prepare them for job applications.
Instead of using different websites career preparation, The project is focused on creating an easy-to-use system that analyses CVs, compares them with job descriptions and provides useful feedback to improve employability.
The first objective of the project is to develop a secure user management system that allows users to register, log in and manage their personal accounts safely.
Authentication is important because it protects user information and allows users to save previous analyses, career recommendations and generated documents inside their registered own account.
the second important objective is to develop a CV analysis system. The application allows users to upload a PDF CV and enter a job description. The system extracts the text from the uploaded CV, it identifies the matching skills and missing skills, compares them with the job requirements and calculates a match score. The results are displayed in the user interface clearly by displaying matched skills, missing skills, detected CV skills and the missing skills helps users to know what they need to improve their cv. The project also uses AI to for interview preparations questions and to generate cover letters based on the user's CV and selected job role. These features are designed to help users prepare more effectively for job applications and to get them fully prepared for the interviews.
the third objective is to create a responsive and smooth user interface that is simple to navigate. The application includes a user dashboard where users can upload CVs, view previous analyses, and allow the user to access career recommendations and use AI to for interview preparation questions and cover letter generator. good layouts and

interactive components are included to provide users with a smooth experience throughout the application. the main aim of the project is to ensure that all parts of the system work together correctly. The React frontend, FastAPI backend, database and AI services must work together so that users receive accurate results quickly. The aim is to develop a reliable, intelligent and complete career development platform that demonstrates an advanced platform while helping users improve their chances of securing employment.

1.3 Overview of Development Approach

The project was developed using a step-by-step approach each part of the project was created and tested this made it easier to identify problems during development and correct them before moving on to other features. The first stage of the development was focused on building the backend using FastAPI the backend was developed to manage user authentication, receive uploaded CV files, extract text from PDF documents, identify skills and communicate with the database. Once these core functions were completed and tested, additional AI features were added to support CV analysis and career recommendations. The next stage was building the frontend of the application was developed using React with Vite after the backend was developed. A responsive user interface was created to allow users to register, log in, upload their CV, enter a job description and view the analysis results. Additional pages, including the dashboard, settings page and career recommendation pages, were added to improve navigation and provide a better user experience. After the CV analysis feature was completed, additional career support tools was developed. These included AI-generated cover letters and interview practice. Each feature was implemented separately and tested to ensure that it produced relevant and personalised results based on the user's CV and selected job role. These features were integrated into the application to provide users with a complete career development platform.

1.4 Document Structure

The documentation is organised into several chapters, where every chapter is focusing on a different aspect of the project

Chapter 2 presents the background of the project and discusses the main concepts and technologies used for the project. This helps provide context for the decisions made throughout the project. CV analysis, skills matching, career recommendation systems and the use of Artificial Intelligence in helping people get employed. The chapter also considers related systems in order to provide context for the development of CareerPilotAI.

Chapter 3 discusses the methodology used to develop the project. It explains the development approach, project requirements, technology used choices and the methods used for CV processing, skill identification, job matching and career recommendations. It also explain the method used in testing and debugging during development.

Chapter 4 focuses on the design and specification of CareerPilot AI. It describes how different components of the project are structured and how they interact with each other. It explains how the React frontend, FastAPI backend, database and external AI services interact. The chapter also explains the design of the user interface, CV processing, skills-matching process, career recommendation functionality.

Chapter 5 describes the implementation of the project. It explains how the main features were developed, including user authentication, PDF CV upload and text extraction, CV skill detection, job-description skill extraction, identification of matched and missing skills, calculation of the job match score and career recommendations. this chapter also explains the React and Vite frontend, frontend styling, frontend-to-backend communication, interview practice and AI-assisted cover-letter generation.

Chapter 6 presents the testing and evaluation of CareerPilot AI. including the challenges encountered and how they were resolved. This includes testing the backend API, CV parser, skills-matching functionality, match-score calculation, career recommendations, frontend components, system integration and AI-supported features.

Finally, Chapter 7 is the conclusions and discusses possible improvements and future work that could be carried out to extend the project.

Chapter 2: Background

2.1 Development Tools and Technologies Used

CareerPilot AI was developed as a full-stack web application using a combination of frontend, backend, database and Artificial Intelligence technologies.
The backend was created using Python with FastAPI and the Frontend was created using React with Vite. Creating different folder for backend and frontend makes the project really neat and allows the project to be tested easily.
React was used to create the frontend because is a JavaScript library used for building user interfaces from reusable components which makes it suitable for CareerPilot AI because the application contains several pages and features, including CV analysis, job comparison, career recommendations, learning roadmaps, interview practice and cover-letter generation. using separate React components also made the frontend easier to arrange and update during development , test and update during development. Vite was used as the development tool for the React frontend. vite is used for the development environment used to run and build the frontend application and CSS was also used to control the layout and visual appearance of the user interface.
Python was used as the main programming language for the backend of the project. It process CV information, identify skills and perform the main application logic.
Python with FastAPI was used to create the backend API and connect it with the React frontend. so when a user uploads a CV or requests information from the system, the frontend sends the request to the backend where it is processed before the result is returned and displayed to the user.
Another important technology used in CareerPilot AI was the ESCO dataset. ESCO contains information about occupations and the skills associated with them. It was

used to compare skills identified from a user's CV with skills required for different occupations. This helped the system identify matched skills, missing skills and suitable career recommendations.
SQLite was used to store application data, while CSS and Tailwind CSS were used to design and organise the appearance of the user interface. Vite was used to run and develop the React frontend, and Git and GitHub were used for version control and storing the project repository. Using these technologies together helped keep the frontend, backend, data processing and AI functionality organised into different parts of the system. Gemini API was also integrated into the project to provide Artificial Intelligence features like generate interview questions and also generate cover letters. The AI service was connected through the Python backend so that the API key could be kept outside the frontend code.

2.2 CV Analysis and Matched Skills

CV analysis and matched skills is an important aspect of CareerPilot AI because it allows the application to understand a user's existing skills and compare them with the skills required for the career the user entered in their description. the system allows a CV to be uploaded and uses the information contained in the cv
When a user uploads a CV, the file is sent from the React frontend to the Python backend for processing. The system extracts the important text from the CV and analyses the cv to identify skills that matches their career path. This information is then prepared for the next stage of the analysis, where the identified skills are compared with occupational information stored in the ESCO dataset.
ESCO is the data source used because it provides structured information about occupations and the skills associated with them. The skills identified from the user's CV are compared with the skills connected to different occupations. From this comparison, CareerPilotAI can determine which skills the user already has and which skills are missing for a particular career. This makes the result more useful than simply displaying a list of skills because the user can see how their current skills are relate to their career paths. The matched skills process was used to support career recommendations within the application.

Occupations that matches the identified skills will be recommended as suitable career recommendation, while the missing skills can be studied and completed using the learning roadmap. Development of the CV analysis and matched skills created a connection between the information provided by the user and the career information available within the system.

2.3 Career Recommendation Systems

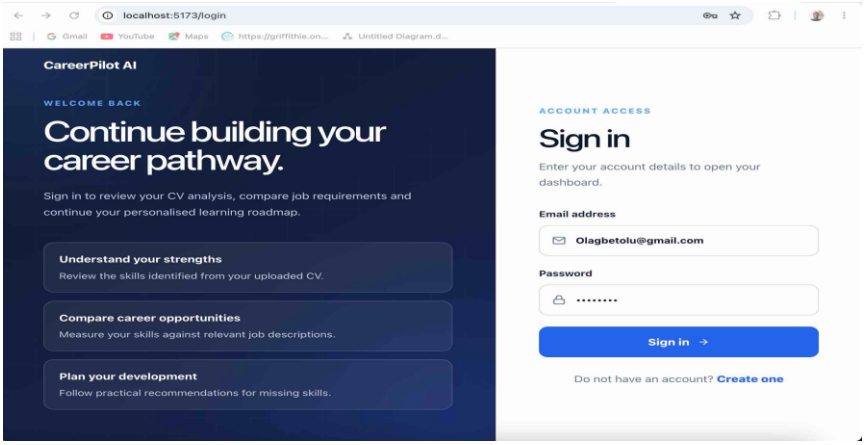
The career recommendation in CareerPilot AI helps users identify alternative careers that are related to the skills they have in their CV. Instead of using all their time on trying to acquire the missing skills from the career path, the system uses the results from the CV analysis and their matched skills to process an alternative career recommendations that are more relevant to the user. The career recommendation process works after the user's CV has been analysed and the required skills for the career has been identified. These skills are compared with the occupation and skills information provided from the ESCO dataset. The system checks how the user's identified skills are relate to the skills associated with different occupations. Other careers that have the same matched skills with the user skills will be recommended to the user. The career recommendation results are connected with other features of CareerPilot AI. This creates a connection between CV analysis, matched skills and the recommendations presented by the application. developing the career recommendation makes the project better because the alternative career suggestions are based on information obtained from the user's own CV and the occupational data used by the system. The purpose of the career recommendation system is to help users identify alternative career opportunities that are relevant to their existing skills and provide guidance towards possible career paths.

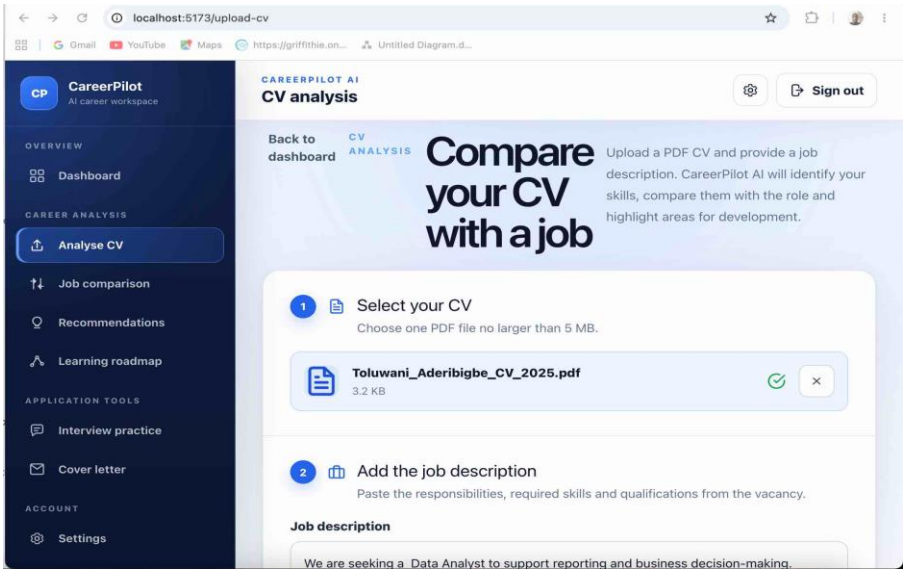
2.4 User Interface and Project Flow

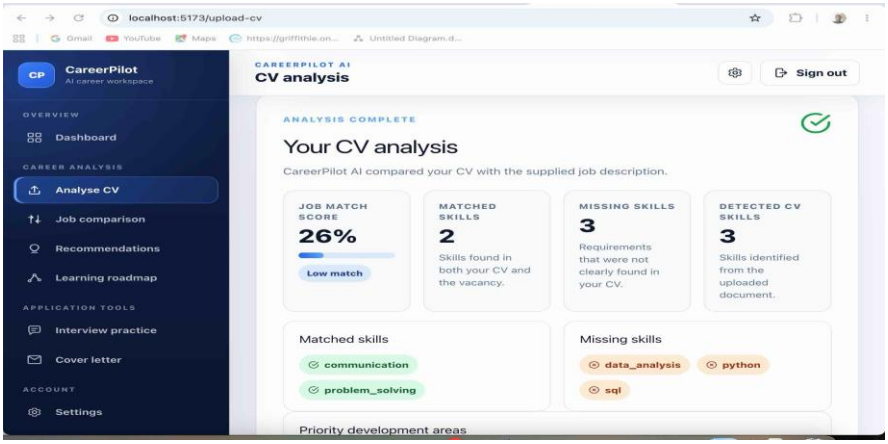
The user interface of CareerPilot AI allows users to interact with the different features of the system. The user interface was designed to provide a simple and

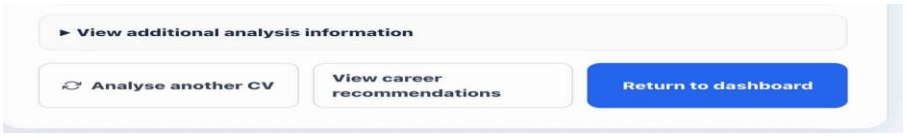
organised way for users to navigate through the application. The project has different pages such as CV upload, career recommendations, job comparison, learning roadmaps, interview practice and cover-letter support. There is also a dashboard so that the user can navigate to any page of their choice. The user needs to register before they can log in into the application. then after authentication, the user can then access the dashboard and upload a CV, and enters there job description for analysis. The CV information is sent to the backend where the relevant skills are identified and compared with the ESCO occupation and skills data. The results are sent back to the frontend user interface so that the user can see the matched skills, missing skills and career recommendations.

The user interface allows the user to navigate from their analysis results to other pages of CareerPilot AI. An example is after the user see there career analysis result they can navigate to the career recommendation page, they can also navigate to the learning roadmap to learn about there missing skills, practise interview questions and generate cover letter. Creating all this pages makes the application easier to navigate and preventing unnecessary information to be displayed on the UI. React was used to build the user interface creating different pages and reusable components, while CSS was used for styling and to maintain the visual appearance and layout of the application.









2.5 ESCO Occupation and Skills Data

ESCO occupation and skills data provides structured information about different occupations and the skills associated with them. The ESCO data is used as a reference when comparing the skills identified from the user's CV with the skills required for different occupations. CareerPilot AI uses the skills identified from the user's CV and

compares them with the skills associated with occupations in the ESCO data. Which allows the system to identify the matched skills and the missing skills for the user required career.
The ESCO data also supports the alternative career recommendation feature. after identifying the user matched skills, the system can compare the matched skills with other career paths in the dataset and other careers that matches the skills the user has The ESCO data is accessed through the backend of CareerPilot AI during the analysis and career recommendation process. The occupation and skills processing is carried out within the backend, which separates the matching logic from the user interface. The results are then returned to the frontend, where the user can see their matched skills, missing skills and alternative career recommendations.

2.6 Artificial Intelligence in Career-Support Systems

Artificial Intelligence is used in CareerPilot AI to generate interview practice questions and cover letters after the user's career and skills information has been analysed. The career and skills information is extracted from the user's CV analysis and the ESCO data used by the system.
CareerPilot AI integrates the Google Gemini API through the Python backend. The backend sends the required information to the AI service and receives a generated response, which is then sent to the React frontend and displayed to the user. Keeping this process within the backend. AI is used for interview practice and cover letter generation. for interview practice. the system generates questions based on the user's selected career, allowing the user to practise interview questions. AI is also used to generate a cover letter based on the career and the description written by the user this makes the AI component useful for supporting users after their career and skills analysis.

Chapter 3: Methodology

3.1 Development Approach

CareerPilot AI was developed using a step-by-step approach, where the main parts of the application were created and tested gradually. which made the project easier to manage and helped reduce errors during development.

The project started with the basic backend using python with FastApi. The backend was created to receive requests, process information and handle the main application logic. The React frontend was then developed to provide the pages and user interface required to interact with the backend. Features such as user registration and login, CV upload and analysis, skills matching, career recommendations and learning roadmaps were added gradually and tested as they were developed.

The CV analysis and matching skills process was an important stage of development because most of the remaining features on the application depend on the results produced from the user's CV. The CV processing was developed and tested before connecting the results with the ESCO occupation and skills data so that the Career recommendations and learning roadmaps could then use the matched and missing skills produced during the analysis.

Artificial Intelligence features was integrated after the main application structure was developed and tested correctly. The Gemini API was connected through the Python backend and it is used for interview practice questions and cover letter generation and was tested and working correctly. Using step by step approach makes the project easier to identify which part of the application required changes instead of changing the entire system.

3.2 Requirements and Planning

Before the development of CareerPilot AI, the main requirements for the project were identified and divided into different features the requirements were divided into functional and technical requirements.

The main functional requirements included are user registration, login, CV upload, job description input, CV analysis, identify matched skills, identify missing skills,

calculation of a match score, career recommendations, job comparison, learning roadmaps, interview practice and cover-letter generation. Each feature was developed to perform a specific task and also connected with the other parts of the application. Technical requirements was also used during the development of the project. React was used to build the frontend and Python with FastAPI was used for the backend. SQLite was used for persistent application data, ESCO occupation and skills data was used to support skills matching and career recommendations, while the Gemini API was integrated for interview practice and generate cover letters. The project was planned so that the main features could be developed and tested properly before being connected together. This helped to keep the development organised and reduced errors during development. The requirements of CareerPilot AI were connected to the main purpose of the project, which is to help users understand their matched and missing skills and provide useful support for their career development.

3.3 Implementation Process

The implementation process of CareerPilot AI was carried out in different stages. At first to start the project the backend and frontend development tools were first identified the basic structure of how the project was going to be was drawn.
The first stage involved developing the Python backend was developed using FastAPI to receive requests and perform the main processing required by the application. The React frontend was then developed with the main pages, including registration, login and the dashboard.
The next stage is on CV processing and skills analysis. The CV upload feature was connected to the backend so that text from an uploaded PDF CV could be processed, job description entered by the user was then implemented so that the related skills can be identified, the identified cv skills and job requirements were then compared to identify the matched skills, missing skills and a match score.

The identified skills were then compared with occupation and skills information from the ESCO data to determine the user's matched and missing skills. after the CV analysis was tested and working, the career recommendation feature was implemented. The matched skills identified during the analysis is used with the ESCO occupation data to find alternative careers recommendation based on the user skills. The learning roadmap and job comparison features was also developed to help the user understand the skills they were missing.

The next stage involved integrating the Artificial Intelligence features. The Google Gemini API was connected through the Python backend and used to generate interview practice questions and cover letters. The generated results were returned from the backend and displayed in the React frontend.

3.4 Testing and Debugging Approach

Testing and debugging was done every time during the development of the project to ensure that the different features of the application is working correctly. Each feature was tested after implementation before being connected to other parts of the system. This helps to avoid errors and identify where the problem was coming from the React frontend, Python backend or the data being processed.

The user registration and login features were tested to make sure users could create an account to access the application. The CV upload and analysis feature was also tested using different pdf CV and different job description to check that the backend could receive the uploaded file, and process its content so it can return the analysis results to the frontend for the user to see.

The ESCO-based career were also tested to check that the skills identified from the user's CV could be compared with occupation and skills data and used for matched skills, missing skills and alternative career recommendations. The results were reviewed during development to make sure that the information returned by the backend would be displayed in the user interface. The Artificial Intelligence required additional testing to be able to communicate with the Google Gemini API. Interview-

question generation and cover-letter generation were tested to ensure that requests could be sent through the Python backend and that generated responses were returned to the frontend.

3.5 Limitations of the Methodology

One limitation of the development approach used for CareerPilot AI was that developing the application step by step sometimes requires some features to be changed. When a new feature is connected to an existing part of the system, additional changes were sometimes needed to make the different components work together correctly.

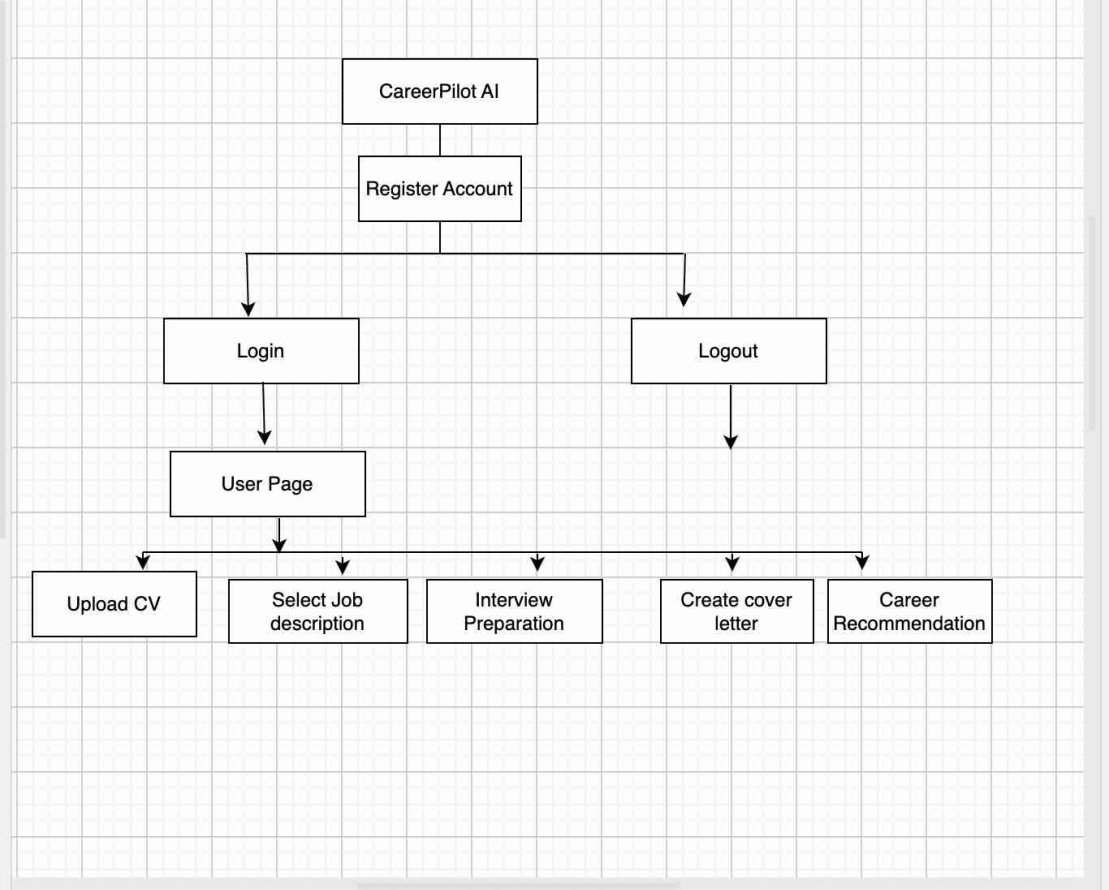
Another limitation was the amount of time for testing required when connecting the frontend and backend. CareerPilot AI contains several features that depend on information being transferred between the React frontend and the Python backend. As more features were added, testing became more time-consuming because changes to one part of the application sometimes affect another part of the system

The use of the data source created limitations. The career analysis and recommendation features depend on ESCO occupation and skills data, while the interview practice and cover-letter features depend on the Google Gemini API. This means that some parts of the application rely on external data that are not build from scratch. an example, the AI features require a valid API configuration and an available connection to the external AI service.

Chapter 4 System Design and Specification

4.1 Overview of System Architecture

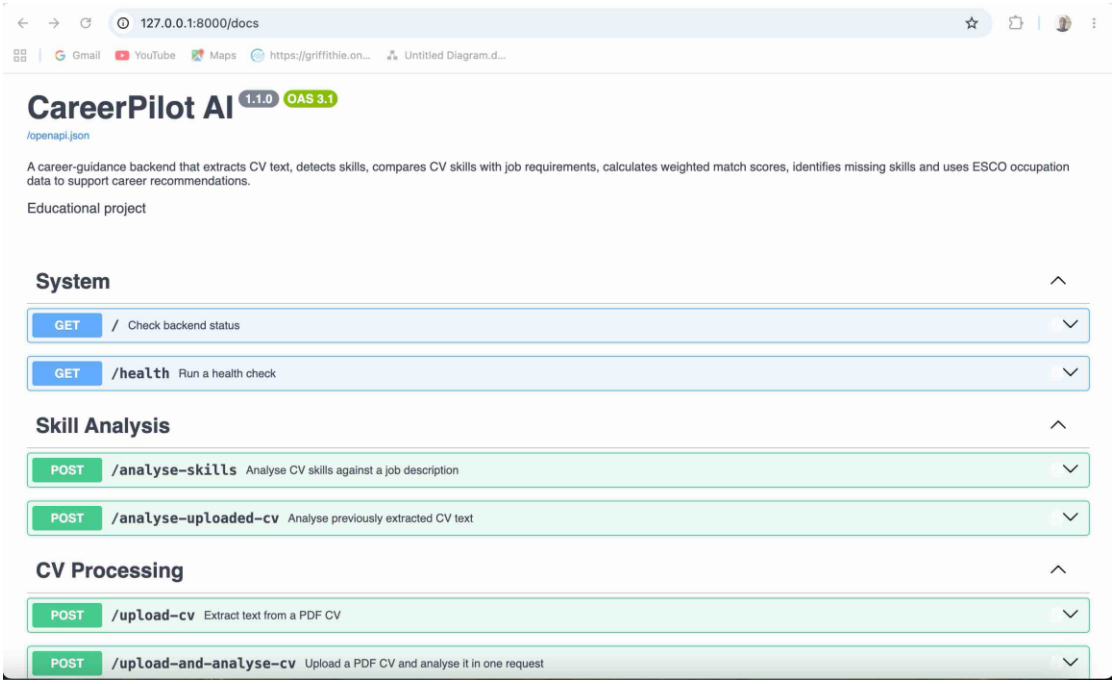
The system architecture of CareerPilot AI is divided into frontend, backend, database and Artificial Intelligence components. separating these components allows every part of the application to perform a specific task while working together as one system. The frontend was developed using React with Vite, is used for the user interface of the application. It allows users to register, log in, upload a CV, enter a job description and access features such as CV analysis, career recommendations, learning roadmaps, interview practice and cover-letter generation., while the Python backend was developed using FastAPI. The backend is responsible for processing requests, CV text extraction, skills analysis, career recommendations, database operations and communication with external services. When a user performs an action that requires processing, the React frontend sends a request to the backend and the processed result is returned to the frontend to be displayed to the user. SQLite provides persistent storage for the application, while ESCO occupation and skills data is used to identify the matched and missing skills and also give the user an alternative career recommendations. The Google Gemini API was accessed through the backend to generate interview practice questions and generate cover letter.

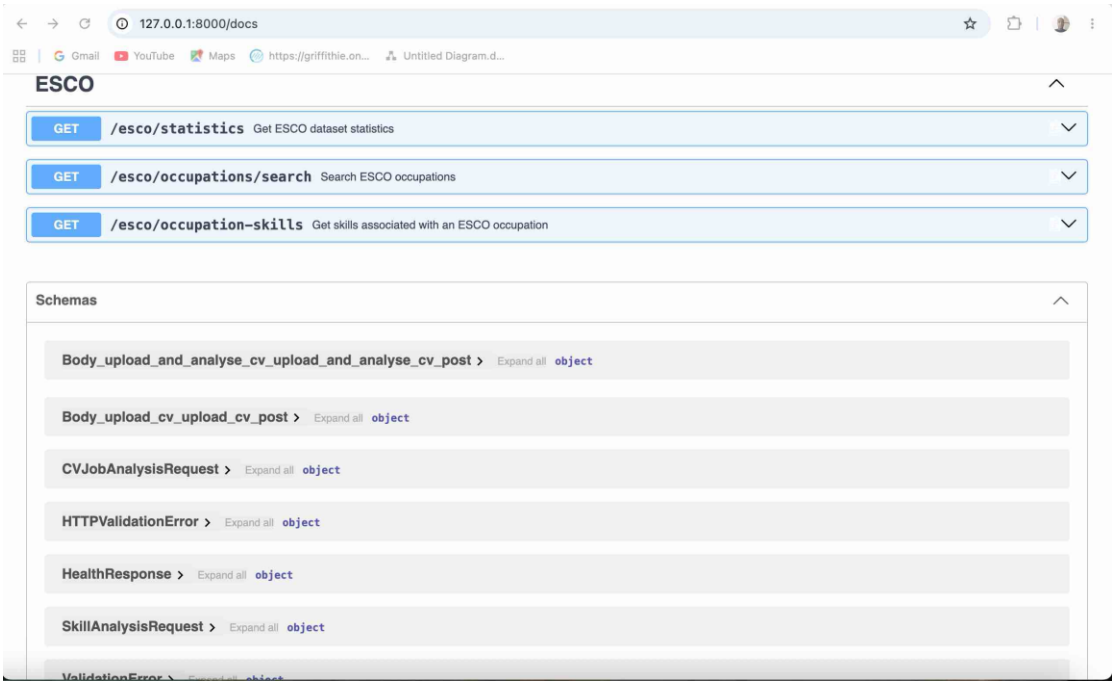


4.2 Backend Design

The backend of CareerPilot AI was developed using Python with FastAPI and it is responsible for processing requests received from the React frontend and return the required information to the user interface. The backend controls the CV processing and compares the identified skills with ESCO occupation and skills data to decides the matched and missing skills. It also provides an alternative career recommendation and also communicate with the SQLite database. Artificial Intelligence features are also controlled through the backend. Requests for interview questions and cover-letter generation are sent to the Google Gemini API, and the generated responses are returned to the frontend. Keeping the backend separate from the user interface makes

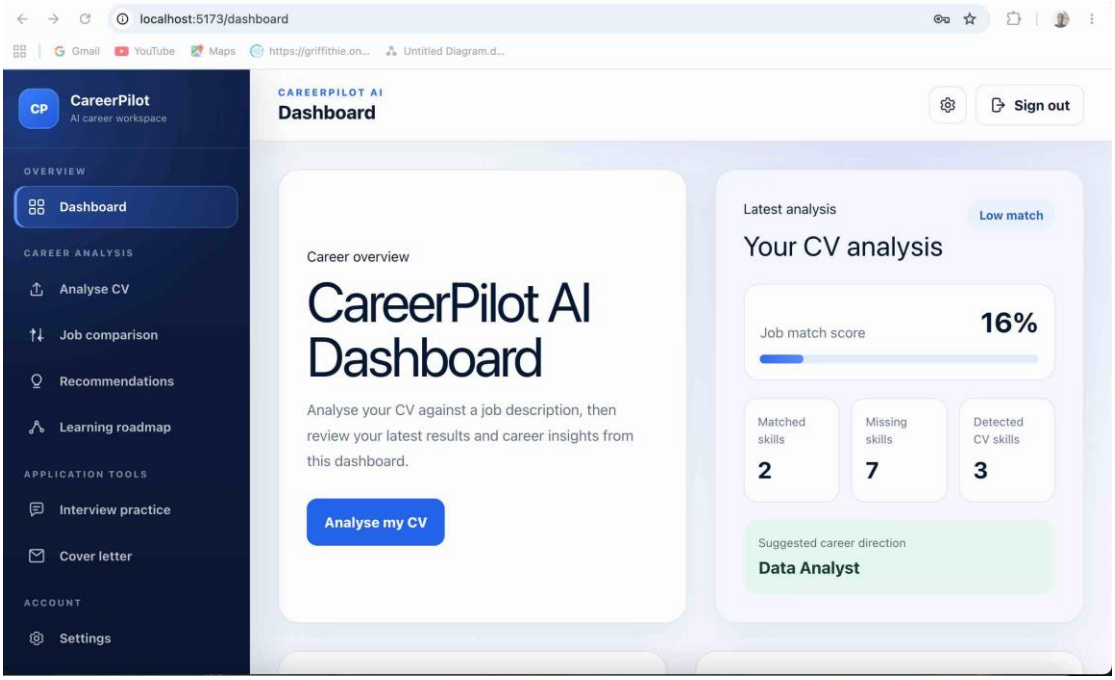
the application easier to organise and maintain.

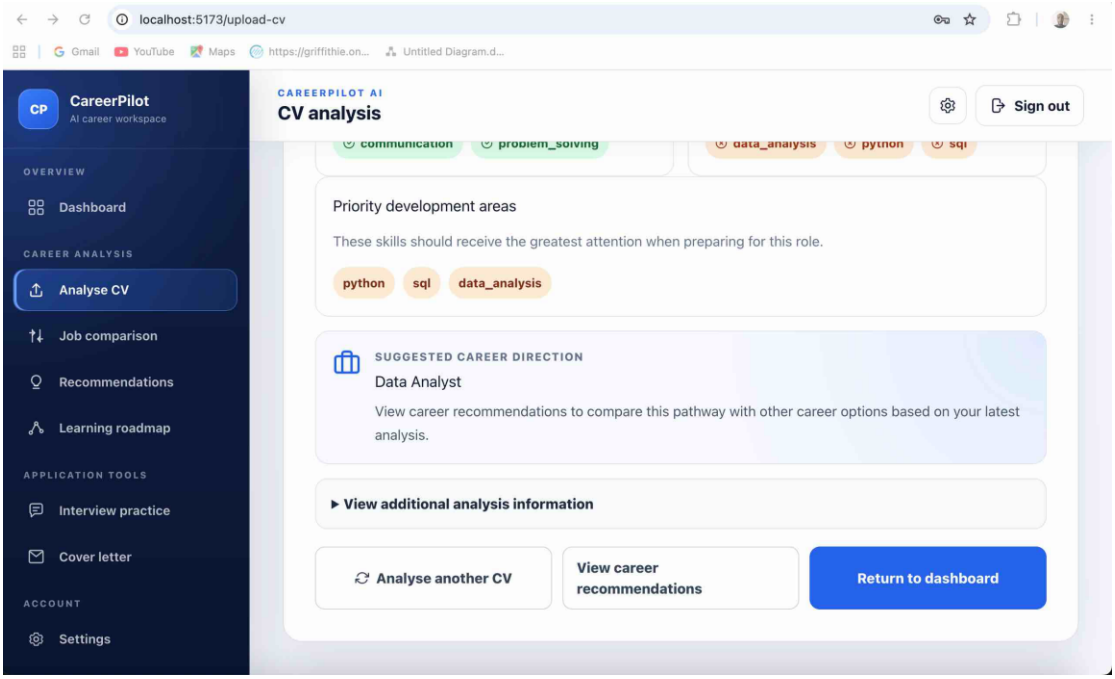




4.3 Frontend Design

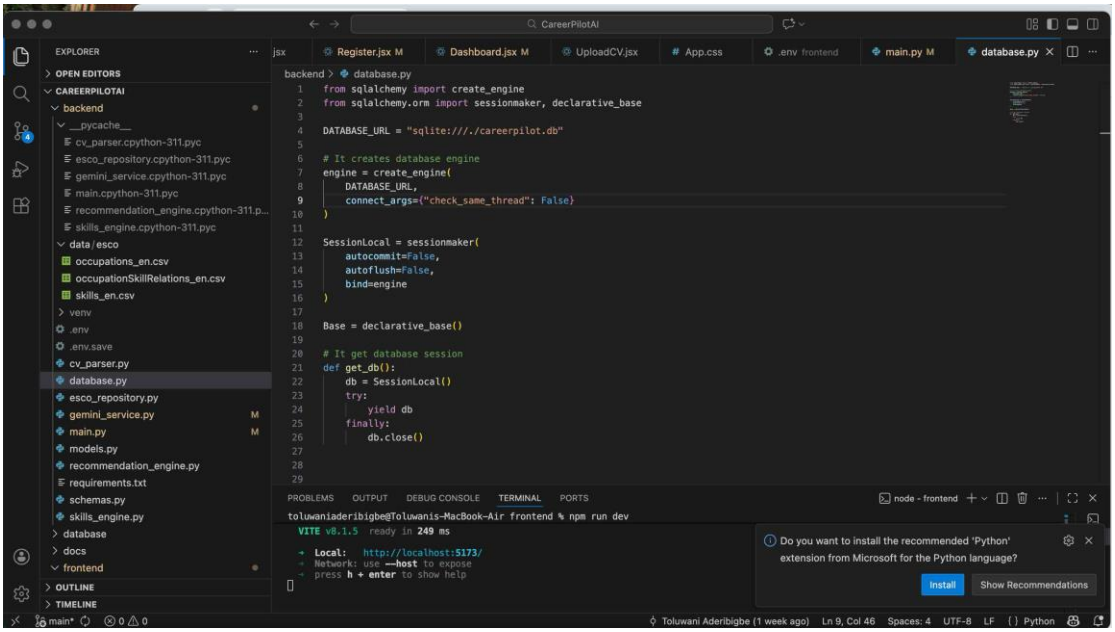
The frontend of CareerPilot AI was developed using React with Vite it provides the user interface so the user can interact with the application. It is organised into different pages and reusable components, which includes registration and login, dashboard, CV analysis, career recommendations, job comparison, learning roadmaps, settings, Not Found, interview practice and cover letter generation pages. the user can also navigate between these pages using the sidebar after the user has logged in.
The frontend is responsible for collecting user input and displaying requests returned from the backend. an example when a CV is uploaded, the information is sent to the backend and the analysis results, including matched and missing skills, are returned and displayed to the user. React components help keep the user interface organised and reduce repetition, while CSS is used to control the layout and appearance of the application. This design keeps the user interface separate from the backend processing while providing clear navigation between the different features.

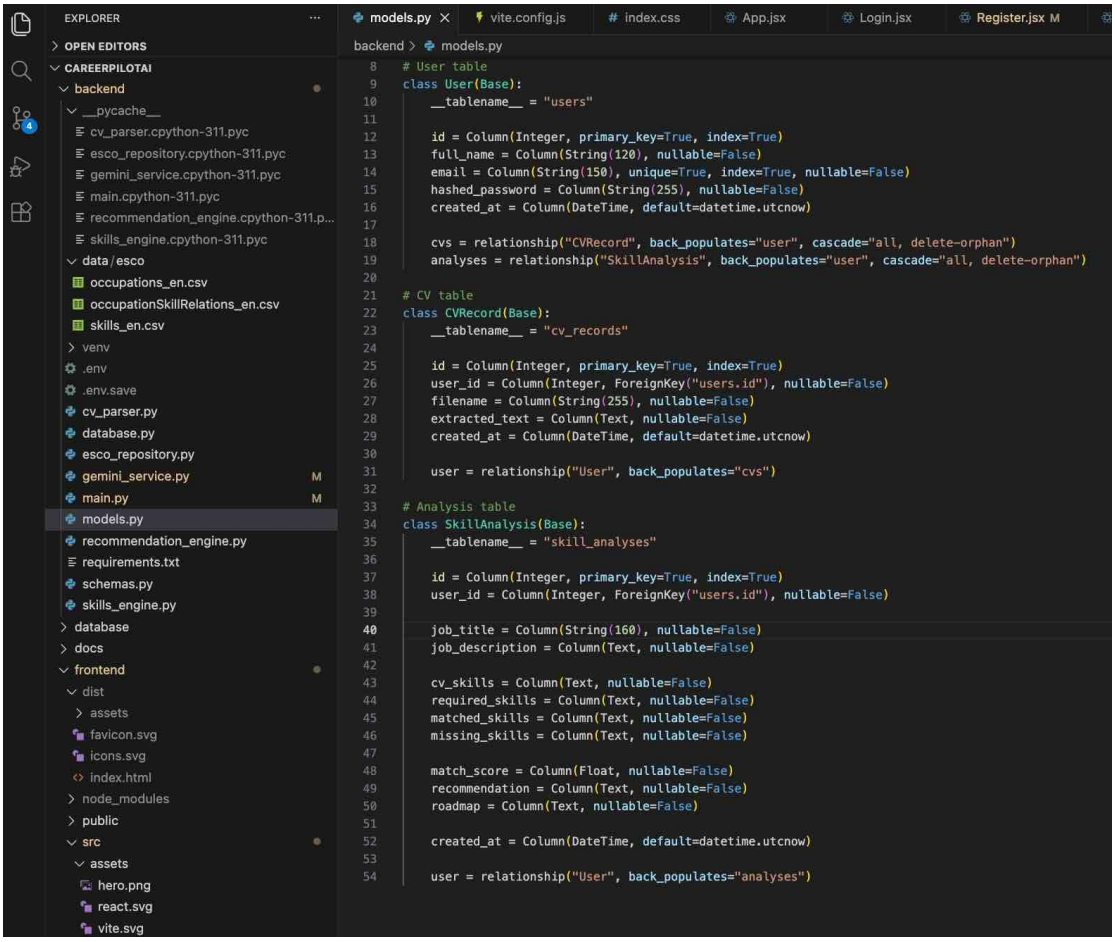




4.4 Database Design

The database use for CareerPilot AI uses SQLite to provide persistent storage for application. SQLite was used because it is lightweight and can be integrated with the Python backend without requiring a separate database server, making it suitable with the requirements of the project. The database is created in the Python backend when information needs to be stored or retrieved, the frontend sends a request to the backend, which performs the required database operation and returns the result. This keeps the database logic separate from the user interface. The database stores a registered user account details needed for authentication, allowing users to log in and access the application. This design provides organised and persistent storage while keeping database operations separate from the main user interface.


CareerPilotAI
EXPLORER
OPEN EDITORS
CAREERPILOTAI
backend
__pycache__
cv_parser.cpython-311.pyc
esco_repository.cpython-311.pyc
gemini_service.cpython-311.pyc
main.cpython-311.pyc
recommendation_engine.cpython-311.p...
skills_engine.cpython-311.pyc
data / esco
occupations_en.csv
occupationSkillRelations_en.csv
skills_en.csv
venv
.env
.env.save
cv_parser.py
database.py
esco_repository.py
gemini_service.py M
main.py M
models.py
recommendation_engine.py
requirements.txt
schemas.py
skills_engine.py
database
docs
frontend
OUTLINE
TIMELINE
jsx
Register.jsx M
Dashboard.jsx M
UploadCV.jsx
App.css
.env frontend
main.py M
database.py
backend > database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
DATABASE_URL = "sqlite:///./careerpilot.db"
# It creates database engine
engine = create_engine(
DATABASE_URL,
connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(
autocommit=False,
autoflush=False,
bind=engine
)
Base = declarative_base()
# It get database session
def get_db():
db = SessionLocal()
try:
yield db
finally:
db.close()
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
node - frontend
toluwaniaderibigbe@Toluwanis-MacBook-Air frontend % npm run dev
VITE v8.1.5 ready in 249 ms
Local: http://localhost:5173/
Network: use --host to expose
press h + enter to show help
Do you want to install the recommended 'Python' extension from Microsoft for the Python language?
Install
Show Recommendations
main*
0 0
Toluwani Aderibigbe (1 week ago)
Ln 9, Col 46
Spaces: 4
UTF-8
LF
Python

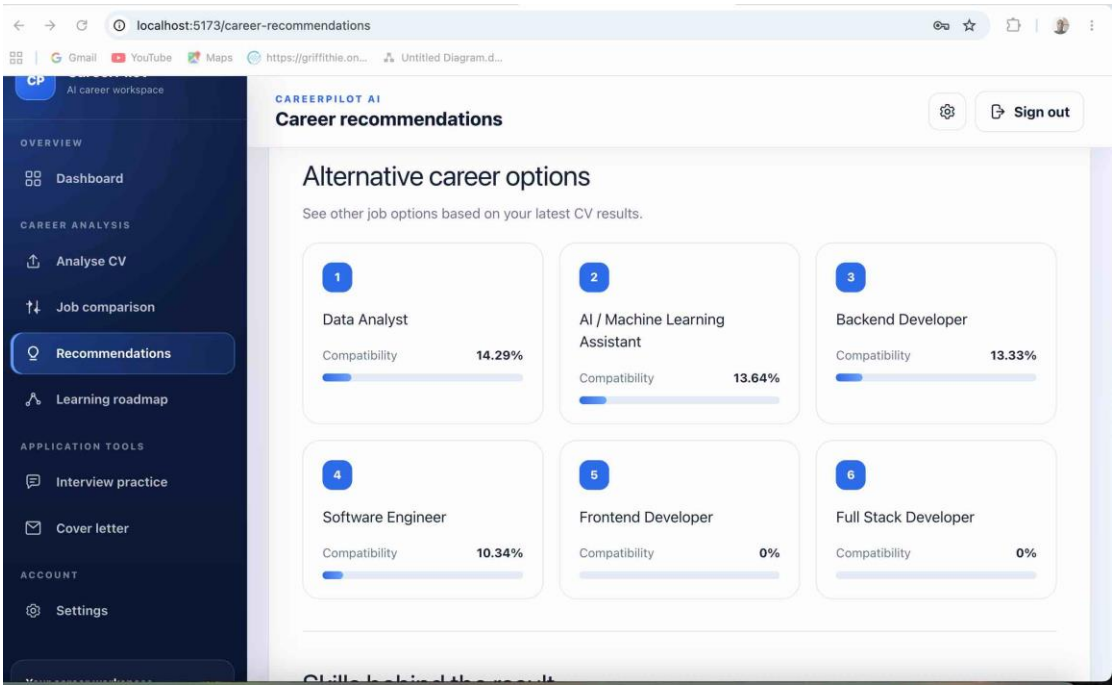

EXPLORER
OPEN EDITORS
CAREERPILOTAI
backend
__pycache__
cv_parser.cpython-311.pyc
esco_repository.cpython-311.pyc
gemini_service.cpython-311.pyc
main.cpython-311.pyc
recommendation_engine.cpython-311.p...
skills_engine.cpython-311.pyc
data / esco
occupations_en.csv
occupationSkillRelations_en.csv
skills_en.csv
venv
.env
.env.save
cv_parser.py
database.py
esco_repository.py
gemini_service.py M
main.py M
models.py
recommendation_engine.py
requirements.txt
schemas.py
skills_engine.py
database
docs
frontend
dist
assets
favicon.svg
icons.svg
index.html
node_modules
public
src
assets
hero.png
react.svg
vite.svg
models.py
vite.config.js
index.css
App.jsx
Login.jsx
Register.jsx M
backend > models.py
# User table
class User(Base):
__tablename__ = "users"
id = Column(Integer, primary_key=True, index=True)
full_name = Column(String(120), nullable=False)
email = Column(String(150), unique=True, index=True, nullable=False)
hashed_password = Column(String(255), nullable=False)
created_at = Column(DateTime, default=datetime.utcnow)
cvs = relationship("CVRecord", back_populates="user", cascade="all, delete-orphan")
analyses = relationship("SkillAnalysis", back_populates="user", cascade="all, delete-orphan")
# CV table
class CVRecord(Base):
__tablename__ = "cv_records"
id = Column(Integer, primary_key=True, index=True)
user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
filename = Column(String(255), nullable=False)
extracted_text = Column(Text, nullable=False)
created_at = Column(DateTime, default=datetime.utcnow)
user = relationship("User", back_populates="cvs")
# Analysis table
class SkillAnalysis(Base):
__tablename__ = "skill_analyses"
id = Column(Integer, primary_key=True, index=True)
user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
job_title = Column(String(160), nullable=False)
job_description = Column(Text, nullable=False)
cv_skills = Column(Text, nullable=False)
required_skills = Column(Text, nullable=False)
matched_skills = Column(Text, nullable=False)
missing_skills = Column(Text, nullable=False)
match_score = Column(Float, nullable=False)
recommendation = Column(Text, nullable=False)
roadmap = Column(Text, nullable=False)
created_at = Column(DateTime, default=datetime.utcnow)
user = relationship("User", back_populates="analyses")

4.5 CV Parsing System Design

The CV parsing system in CareerPilot AI is responsible for processing an uploaded CV and extracting the text needed for analysis. When the user uploads their CV through the React frontend user interface, the file is sent to the Python backend for processing. The backend extracts the CV text and prepares it so that relevant skills can be identified. The identified skills are then used in the skills-matching process, where they are compared with the skills required for the user's selected career using ESCO occupation and skills data. Skills identified in both sources are presented as matched skills, while required skills that are not identified in the user's CV are presented as missing skills. The results are then returned to the frontend and displayed to the user. These results are also used for the alternative career recommendations and the learning roadmap. creating the CV parsing process within the backend separates the processing logic from the user interface and provides a good user interface result.

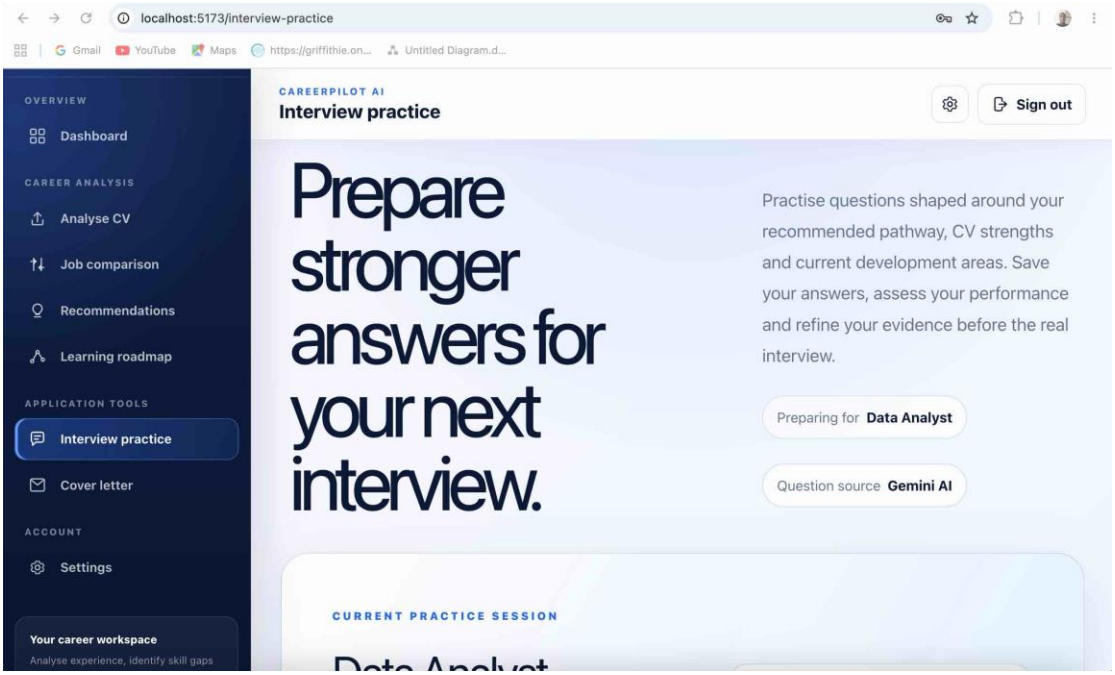
4.6 Career Recommendation System Design

The career recommendation component was designed to use skills identified during CV analysis to suggest alternative career paths that may be reletad to the user career. The recommendation process is performed within the Python backend. after the CV analysis has been completed, the user's identified skills are compared with occupation and skills information from the ESCO data. Careers that requires skills that user has on their cv are been considered as the alternative career path for the user. The career recommendation process is controlled by the Python backend, while the results are sent to the React frontend and then displayed to the user.



4.7 AI Integration Design

Artificial Intelligence is integrated into CareerPilot AI through the Google Gemini API to support interview practice and cover letter generation. The AI service is accessed through the Python backend rather than directly from the React frontend. when a user requests interview questions or a cover letter, the required information is sent from the frontend to the FastAPI backend, then the backend prepares the request and communicates with the Gemini API. The generated response is then received by the backend and returned to the frontend to be displayed to the user. This design separates the AI processing from the user interface and allows the frontend, backend and external AI service to work together efficiently.


localhost:5173/interview-practice
Gmail
YouTube
Maps
https://griffithie.on...
Untitled Diagram.d...
OVERVIEW
Dashboard
CAREER ANALYSIS
Analyse CV
Job comparison
Recommendations
Learning roadmap
APPLICATION TOOLS
Interview practice
Cover letter
ACCOUNT
Settings
Your career workspace
Analyse experience, identify skill gaps
CAREERPILOT AI
Interview practice
Sign out
Prepare stronger answers for your next interview.
Practise questions shaped around your recommended pathway, CV strengths and current development areas. Save your answers, assess your performance and refine your evidence before the real interview.
Preparing for Data Analyst
Question source Gemini AI
CURRENT PRACTICE SESSION

Chapter 5 Implementation

5.1 Overview of Implementation

CareerPilot AI was implemented as a full-stack web application with separate frontend and backend components. The frontend was developed using React with Vite, while the backend was developed using Python with FastAPI.
SQLite was used for persistent application data, ESCO occupation and skills data is used for the skills-analysis and career-recommendation features, and the Google Gemini API was integrated for AI features such as interview practise questions and cover letter generation. The project was developed step by step and was tested before going to the next part before being connected. which includes user registration and login, CV upload and text extraction, skills analysis, matched and missing skills, career recommendations, job comparison, learning roadmaps, interview practice and cover letter generation. The React frontend sends requests to the FastAPI backend, where user information is processed before the results are returned and displayed to the user.

5.2 Backend Implementation Using Python and FastAPI

The backend of CareerPilot AI was implemented using Python with FastAPI. the FastAPI was used to create the API endpoints used to communicate between the React frontend and the main application logic. the backend contains the processing required by the different features of the application. For CV analysis, it receives the uploaded CV and job description from the frontend, the uploaded CV is processed so that its text can be extracted and is used for skills analysis. The identified skills are compared with ESCO occupation and skills data to know the matched skills and missing skills and to also give the user an alternative career recommendations. The backend controls the communication with the SQLite database for persistent data storage. Google Gemini API requests are processed through the backend so when the user try to practice for interview or try to generate cover letter so the generated responses are then returned to the React frontend and displayed to the user. FastAPI also provides interactive API documentation, which was used during development for viewing and

testing the implemented endpoints separately. The screenshot below shows the /analyse skills endpoint used to receive CV text and job information for skills analysis.

5.3 Frontend Implementation Using React and Vite

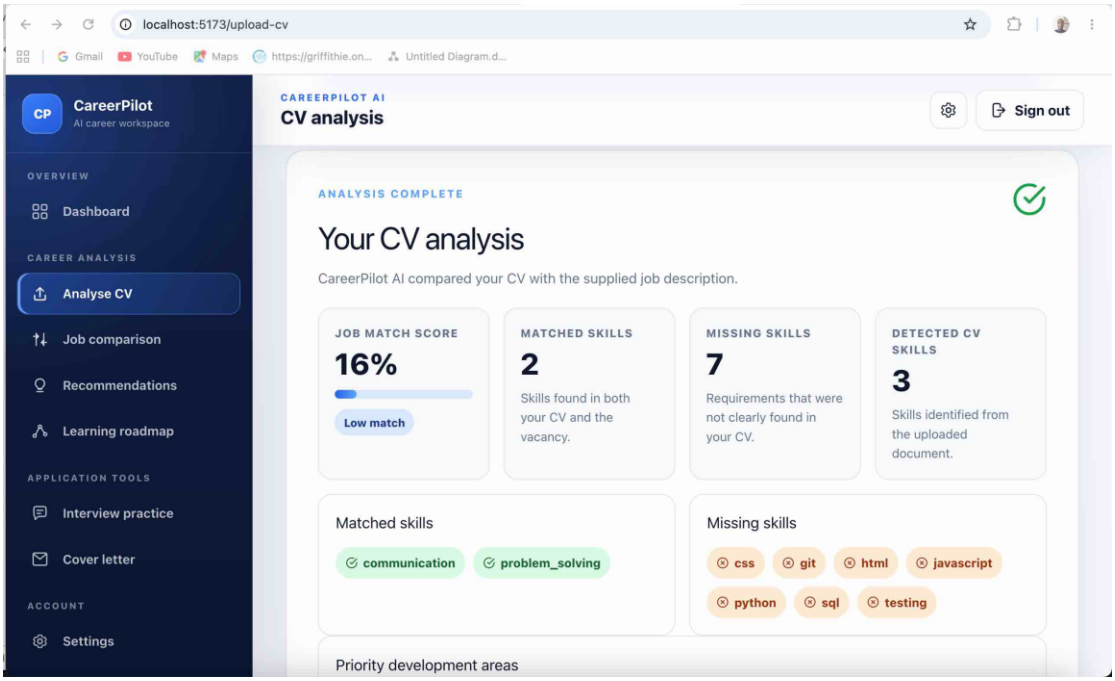
The frontend of CareerPilot AI was implemented using React with Vite. React was used to build the user interface using reusable components and different pages, while and Vite was used to run and develop the frontend application. the pages implemented are Registration, login, dashboard, CV analysis, job comparison, career recommendations, learning roadmap, interview practice, cover-letter generation and settings. and the user can also navigate through all this pages in the application.
The frontend is connected to the FastAPI backend to send and receive information required by the application. an example when a user uploads a CV and enters their job description, the frontend sends the information to the backend and displays the returned analysis to the UI which are the match score, matched skills and missing skills. The user can navigate to another page inside the application. CSS and Tailwind CSS were used for styling and to create a responsive user interface.

5.4 Database Implementation

SQLite was implemented as the persistent database for CareerPilot AI and is accessed through the Python backend. The database implementation uses models for storing user and application information. User records are used for account registration and authentication, while related records allow CV and analysis information to be stored with the user details. needed. The database connection and session configuration were implemented inside the backend so that other parts of the application can read and write persistent information when needed. the frontend sends a request to the backend which performs the required database operation and returns the result this makes the project organised

5.5 CV Processing and Skills Analysis

The CV processing and skills analysis feature was implemented to identify skills from the CV uploaded by the user through the React frontend and sent to the FastAPI backend, where the text content will be extracted and be analysed. The system will then identifies skills to be used in the career analysis process. The identified skills are compared with the skills associated with the career selected by the user using ESCO occupation and skills data. Skills identified in both the user's CV and the selected career description are known as matched skills, while the skills not identified in the cv is known as the missing skills. The system also calculates the match score to let the user know there grade in there selected career the results are returned to the React frontend and displayed to the user. the results are also used for alternative career recommendations and learning roadmaps.



5.6 ESCO Integration and Career Recommendation

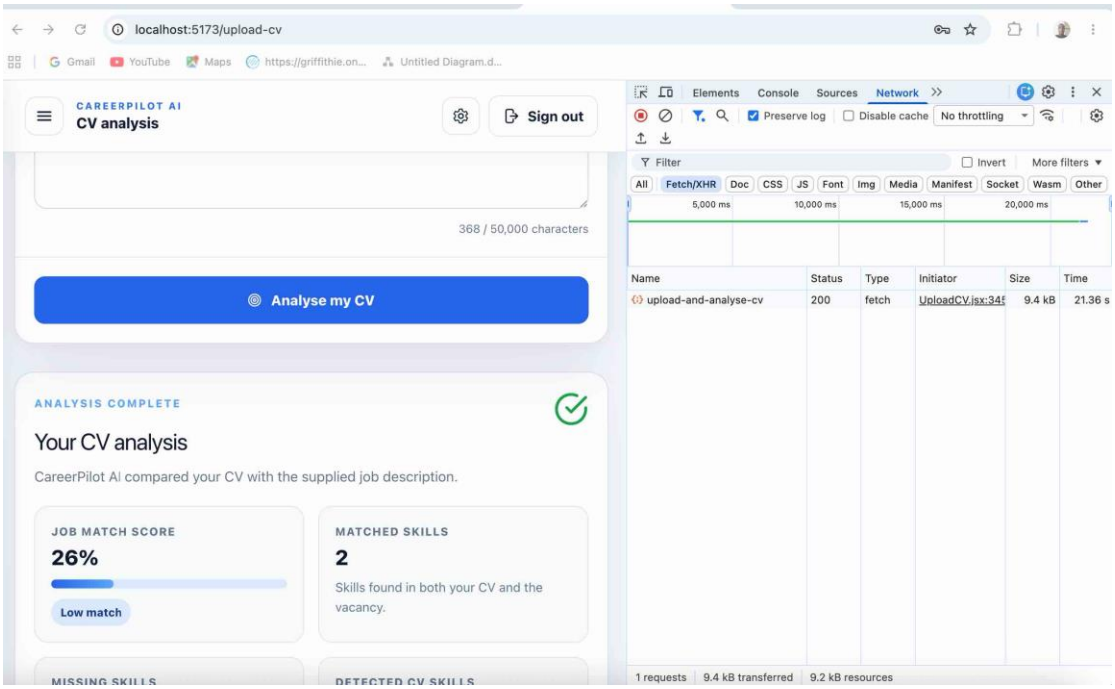
ESCO occupation and skills data was integrated into the Python backend to provide occupational information for the skills analysis and career recommendation features. after skills in the users CV has been identified, the backend then compare this skills with skills associated with different occupations in the ESCO data this data source allows the identification of matched and missing skills and also provides alternative career recommendation feature that uses the user matched skills to search for an alternative occupations based on the related skills. The occupations that requires the same skills are then returned as alternative career recommendations, allowing the user to see alternative careers that are related to the skills already identified from their CV. Then the alternative career recommendation results are sent from the FastAPI backend to the React frontend to be displayed to the user.

5.7 Google Gemini API Implementation

The Google Gemini API was integrated into the project to provide the Artificial Intelligence features in CareerPilot AI this are for interview questions practice and to generate cover letter The API is connected through the Python backend so that AI requests would be controlled without taking the user into the AI API in the user interface. when a user requests interview practice questions, the frontend sends the information to the FastAPI backend. The backend prepares the information required for the AI request and sends it to the Gemini API. The generated response is received by the backend and then returned to the React frontend to be displayed to the user. The cover letter generation also uses the same process. The generated response is returned to the FastAPI backend and then sent to the React frontend so the user can see. the external AI communicate with the backend while allowing the user to access the AI features inside the CareerPilot AI user interface.

5.8 Frontend and Backend Integration

The integration of both the React Frontend and Python Backend was done to make all the different component of the project works together. When a user tries uploading a CV and enters job description and requesting to analyse cv the frontend sends the information to the backend. The FastAPI backend processes the request and returns the result to the frontend UI so the user can see. the integration was tested to ensure that information are sent and received correctly between the two parts of the application. this are all what was tested Registration, Login, CV uploads, Job description, analysis results, Job Comparison, Learning road map, career recommendations, and responses generated through the Google Gemini API. all the errors were identified and fixed during testing. and are connected to make the complete application.



5.9 Testing, Challenges and Problem Solving

During the implementation of CareerPilot AI, every part of the project was tested after building them to ensure that it working correctly before been connected with other parts of the application. This made it easier to identify errors testing were done on all this pages user registration, login, CV upload and processing, skills analysis, career recommendations, the frontend sending and backend receiving, and the Google Gemini API features.

The main problem faced was was making sure that information extracted from an uploaded CV could be processed correctly and used for skills analysis. by the Python backend and that the identified skills can be compared with the ESCO occupation and skills data. This was done by testing different CVs and checking the analysis results. the Frontend and backend were also tested to ensure that requests from the React interface were correctly received and processed by the FastAPI backend. The Google Gemini API requires extra time and much more testing to ensure that interview questions and cover letters were generated by the AI and are returned correctly to the frontend. during the development errors were identified by checking backend responses and testing every part separately before connecting them to the application.

5.10 Summary of Implementation

The implementation of CareerPilot AI involved developing and integrating the main components required for the application. The frontend was implemented using React with Vite, while Python with FastAPI was used for the backend and SQLite for persistent data storage. CV processing and ESCO occupation and skills data were used to identify matched and missing skills and to provide alternative career recommendations. The Google Gemini API was integrated through the backend to provide interview practice and cover-letter generation. The frontend and backend were connected and tested to ensure that information could be processed and returned correctly to the user. CareerPilot AI can be run locally by starting the FastAPI backend and React frontend separately, allowing the components to communicate and to make the application function.

Appendix: Installation, Setup and User Guide

The appendix is the instructions needed to set up, run and use CareerPilot AI in a local development environment. CareerPilot AI is a full-stack web application made up of a Python with FastAPI backend and a React with Vite frontend. and this is the link to GitHub repository. https://github.com/Toluwani9-ai/CareerPilot-AI

Requirements and Setup

Before running the application CareerPilot AI, the required Python and Node.js dependencies should be installed. The project contains separate backend and frontend directories, and both components must be running for the complete application to operate.

Running the Backend

Open the first terminal to run the backend. You'll need to move into the backend folder, then activate your Python virtual environment, and start Uvicorn. This are what you need to run in the terminal.

cd ~/CareerPilotAI/backend

source venv/bin/activate
uvicorn main:app --reload

When the backend starts successfully, the terminal shows this message:
Application startup complete.
Uvicorn running on http://127.0.0.1:8000 then you can access the FastAPI at:
http://127.0.0.1:8000/docs

Running the Frontend

Use the second terminal to start the frontend. Just go to the frontend folder and start the Vite development server.
cd ~/CareerPilotAI/frontend
npm run dev

When the frontend is on then Vite will show this local web address
http://localhost:5173 You can then open the app in your web browser

Frontend and backend communication

The React frontend runs on port 5173, and the FastAPI backend runs on port 8000. When a user logs in, creates an account, uploads a cv, or runs an analysis, the frontend sends a message to the backend. The backend handles the request, sends back the data, and React

Chapter 6 Testing and Evaluation

6.1 Overview of Testing

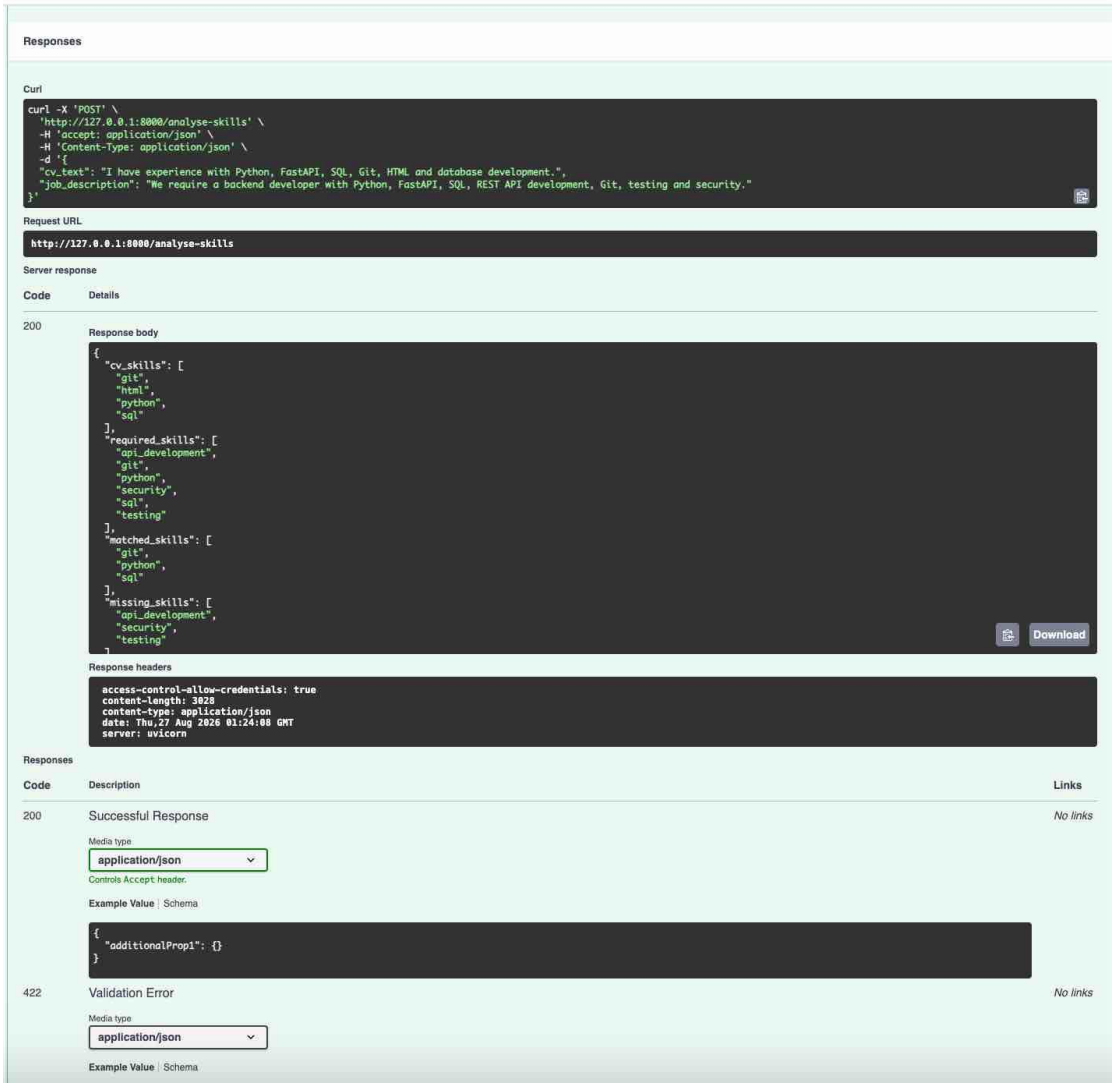
Testing was carried out throughout the development of CareerPilot AI to ensure that every part of the project is working correctly before connecting them together. The testing focused on the main functions of the system, including user registration and login, CV processing, skills analysis, career recommendations, frontend and backend communication, and the Artificial Intelligence features.
Different CVs and job descriptions were used to test the analysis process, and the returned results were checked to make sure that matched skills, missing skills and the job match score were displayed correctly. Testing was also carried out between the React frontend and FastAPI backend to make sure that requests were sent, processed and returned the way it supposed to be . the database was tested and the AI features were tested.

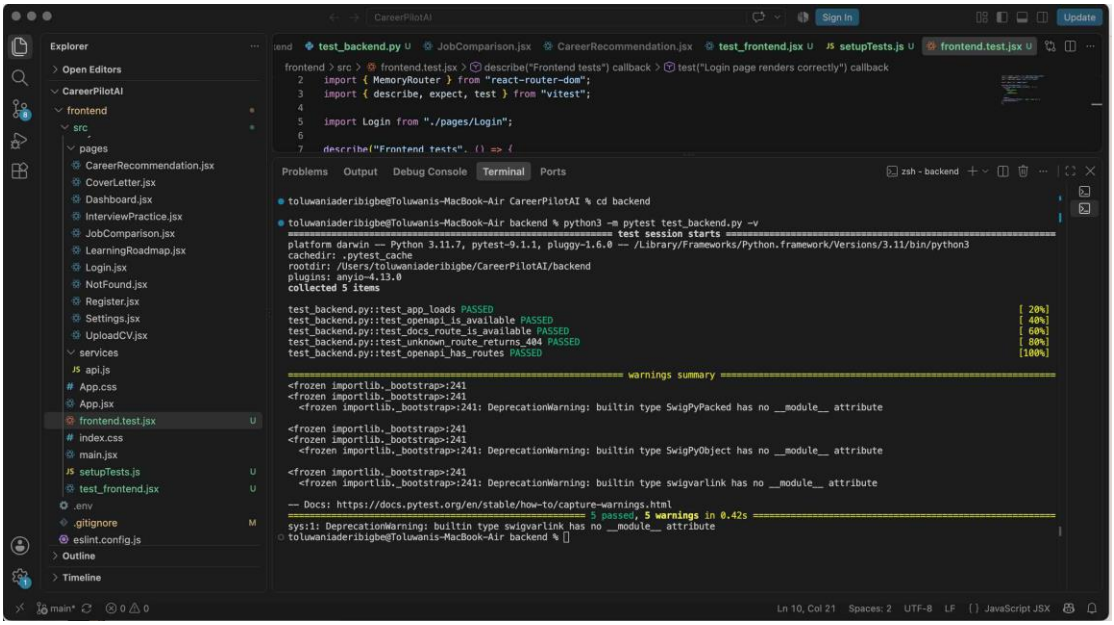
6.2 Backend and API Testing

The FastAPI backend was tested to ensure that the requests sent from the frontend were received and processed correctly. The API functions tested are user authentication, CV processing, skills analysis, career recommendations and the endpoints used by the frontend. The FastAPI /docs interface was also used during development to send requests directly to the backend and check the responses returned by the API. both valid and invalid inputs were tested. registration and login are tested using different user details, while CV analysis was tested using different CVs and job descriptions. The returned responses were checked to make sure that the backend provided the expected results and handled the requests correctly. The skills analysis endpoint was also tested directly using the FastAPI /docs interface. A sample CV text and job description were sent to the /analyse-skills endpoint to check whether the backend could identify and compare the skills correctly. the screenshot below, the request returned a 200 status code**,** which shows that it was successful. The response

contained the skills identified from the CV and job description and separated them into matched and missing skills. Testing was used to identify errors during development. when an error is found, the affected backend function is fixed and tested again this helps to ensure that that the API functions worked correctly before been connected to the frontend.

Five automated backend tests were carried out. The test_app_loads test was carried out to ensure that the FastAPI application could load successfully, The test_openapi_is_available test checked that the OpenAPI schema used to describe the API was available.,The test_docs_route_is_available test checked that the FastAPI /docs route could be accessed successfully. The test_unknown_route_returns_404 test checked the error handling of the backend by requesting a route that does not exist and confirming that a 404 status code was returned, The test_openapi_has_routes test checked that API routes were registered in the OpenAPI schema. The tests were carried out using the pytest testing framework and all five automated tests passed successfully.


Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/analyse-skills' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"cv_text": "I have experience with Python, FastAPI, SQL, Git, HTML and database development.",
"job_description": "We require a backend developer with Python, FastAPI, SQL, REST API development, Git, testing and security."
}'
Request URL
http://127.0.0.1:8000/analyse-skills
Server response
Code
Details
200
Response body
{
"cv_skills": [
"git",
"html",
"python",
"sql"
],
"required_skills": [
"api_development",
"git",
"python",
"security",
"sql",
"testing"
],
"matched_skills": [
"git",
"python",
"sql"
],
"missing_skills": [
"api_development",
"security",
"testing"
]
Download
Response headers
access-control-allow-credentials: true
content-length: 3028
content-type: application/json
date: Thu,27 Aug 2026 01:24:08 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"additionalProp1": {}
}
422
Validation Error
No links
Media type
application/json
Example Value | Schema


Explorer
Open Editors
CareerPilotAI
frontend
src
pages
CareerRecommendation.jsx
CoverLetter.jsx
Dashboard.jsx
InterviewPractice.jsx
JobComparison.jsx
LearningRoadmap.jsx
Login.jsx
NotFound.jsx
Register.jsx
Settings.jsx
UploadCV.jsx
services
api.js
App.css
App.jsx
frontend.test.jsx
index.css
main.jsx
setupTests.js
test_frontend.jsx
.env
.gitignore
eslint.config.js
Outline
Timeline
test_backend.py
JobComparison.jsx
CareerRecommendation.jsx
test_frontend.jsx
setupTests.js
frontend.test.jsx
import { MemoryRouter } from "react-router-dom";
import { describe, expect, test } from "vitest";
import Login from "./pages/Login";
describe("Frontend tests", () => {
Problems
Output
Debug Console
Terminal
Ports
toluwaniaderibigbe@Toluwanis-MacBook-Air CareerPilotAI % cd backend
toluwaniaderibigbe@Toluwanis-MacBook-Air backend % python3 -m pytest test_backend.py -v
test session starts
platform darwin -- Python 3.11.7, pytest-9.1.1, pluggy-1.6.0 -- /Library/Frameworks/Python.framework/Versions/3.11/bin/python3
cachedir: .pytest_cache
rootdir: /Users/toluwaniaderibigbe/CareerPilotAI/backend
plugins: anyio-4.13.0
collected 5 items
test_backend.py::test_app_loads PASSED [20%]
test_backend.py::test_openapi_is_available PASSED [40%]
test_backend.py::test_docs_route_is_available PASSED [60%]
test_backend.py::test_unknown_route_returns_404 PASSED [80%]
test_backend.py::test_openapi_has_routes PASSED [100%]
warnings summary
<frozen importlib._bootstrap>:241
<frozen importlib._bootstrap>:241: DeprecationWarning: builtin type SwigPyPacked has no __module__ attribute
<frozen importlib._bootstrap>:241
<frozen importlib._bootstrap>:241: DeprecationWarning: builtin type SwigPyObject has no __module__ attribute
<frozen importlib._bootstrap>:241: DeprecationWarning: builtin type swigvarlink has no __module__ attribute
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 passed, 5 warnings in 0.42s
sys:1: DeprecationWarning: builtin type swigvarlink has no __module__ attribute
toluwaniaderibigbe@Toluwanis-MacBook-Air backend %

6.3 CV Processing and Skills Analysis Testing

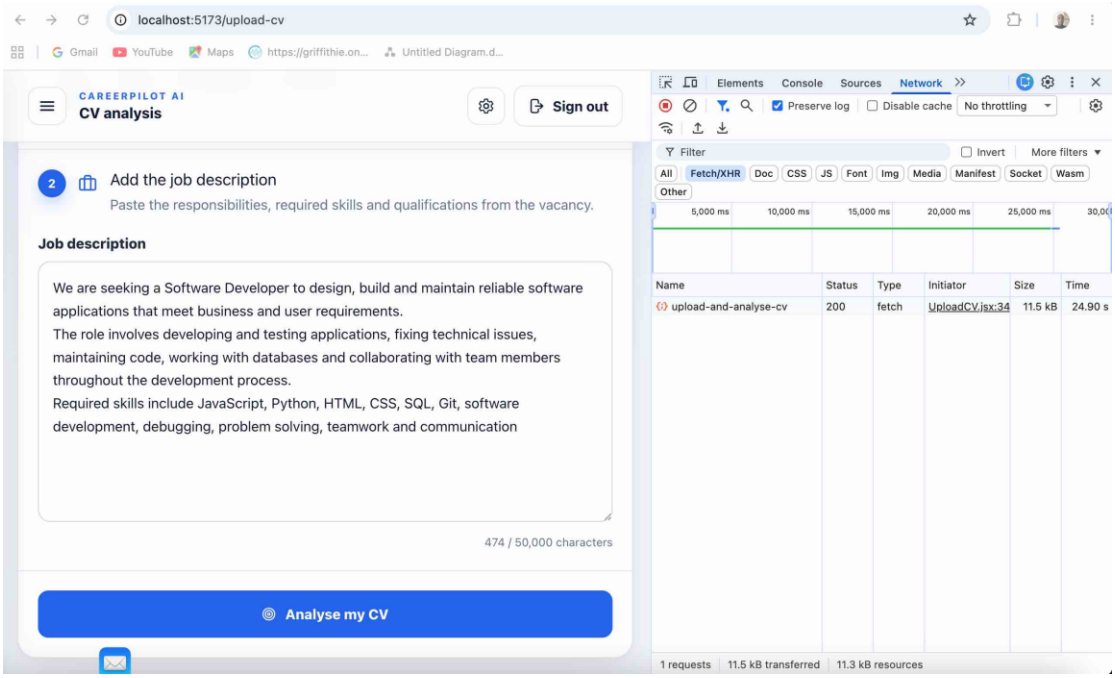
The CV processing and skills analysis features of CareerPilot AI was tested using different pdf CV to check maybe the system could extract and analyse the users information correctly. all the CV was uploaded in the react frontend and processed by the backend to extract its text content and identify relevant skills. The identified skills were then compared with the skills required for the user's selected career. Testing was made to know whether skills found in both sources are displayed as matched skills and the skills missing in the user skills are known as missing skills. The testing confirmed that the CV processing and skills analysis could provide the information required for the career recommendation and learning roadmap features.

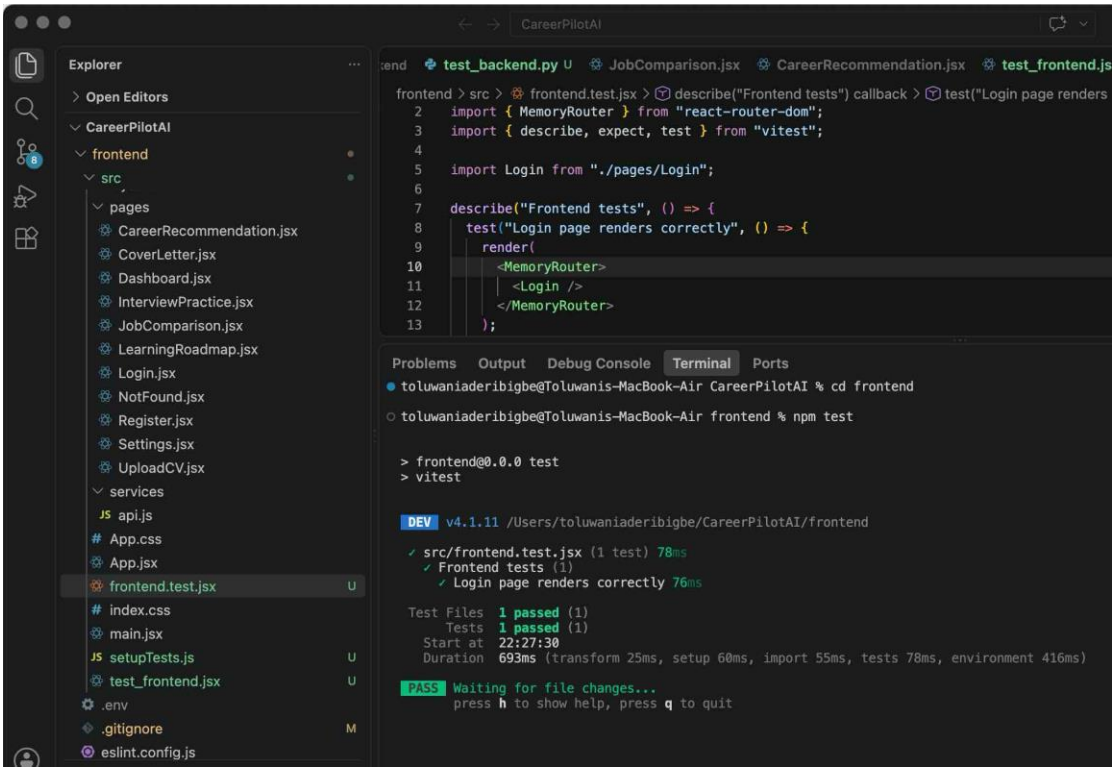
6.4 ESCO and Career Recommendation Testing

The ESCO integration and career recommendation features of CareerPilot AI was tested to ensure that the system is using the occupation and skills data correctly when analysing a user's career paths. different career paths and skill results were tested to check that the system retrieved the relevant ESCO information and compared it with the skills identified from the user's CV. The alternative career recommendation feature was then tested to ensure that alternative careers were suggested based on the user's existing skills required by those other occupations. The recommendations were checked using different CVs and different career description to confirm that the results based on the user's information. Testing ensures that the ESCO data uses skills analysis to produce alternative career recommendations.

6.5 Frontend and Integration Testing

The frontend and integration of CareerPilot AI was tested to ensure that users would be able to navigate to different pages in the application and that the React frontend communicating the right way with the FastAPI backend. The frontend pages which are tested includes registration, login, dashboard navigation, CV upload, career recommendations, job comparison, learning roadmaps, interview practice and cover-letter generation. all the user inputs, buttons and navigations to another page were tested to ensure they are responded correctly and displaying the expected results. testing confirmed that the frontend and backend are working together correctly. Browser developer tools were also used to inspect network requests during integration testing. A successful request to the upload-and-analyse-cv endpoint returned HTTP status code 200**,** providing evidence that the request from the frontend reached the backend successfully and that a response was returned as shown in the screenshot. Automated frontend test was carried out using Vitest and React Testing Library**.** The reason was to check that React component could work correctly and that the user interface will display the expected results. The Login page was used for the test because it first page the user will interact with the test proves that the Login component could render successfully and display the expected Sign In. the test was executed using the npm test command. The result shows that 1 test file passed and 1 test passed**.**


localhost:5173/upload-cv
CAREERPILOT AI
CV analysis
Sign out
2
Add the job description
Paste the responsibilities, required skills and qualifications from the vacancy.
Job description
We are seeking a Software Developer to design, build and maintain reliable software applications that meet business and user requirements.
The role involves developing and testing applications, fixing technical issues, maintaining code, working with databases and collaborating with team members throughout the development process.
Required skills include JavaScript, Python, HTML, CSS, SQL, Git, software development, debugging, problem solving, teamwork and communication
474 / 50,000 characters
Analyse my CV
Elements
Console
Sources
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
5,000 ms
10,000 ms
15,000 ms
20,000 ms
25,000 ms
Name
Status
Type
Initiator
Size
Time
upload-and-analyse-cv
200
fetch
UploadCV.jsx:34
11.5 kB
24.90 s
1 requests
11.5 kB transferred
11.3 kB resources


CareerPilotAI
Explorer
Open Editors
CareerPilotAI
frontend
src
pages
CareerRecommendation.jsx
CoverLetter.jsx
Dashboard.jsx
InterviewPractice.jsx
JobComparison.jsx
LearningRoadmap.jsx
Login.jsx
NotFound.jsx
Register.jsx
Settings.jsx
UploadCV.jsx
services
api.js
App.css
App.jsx
frontend.test.jsx
index.css
main.jsx
setupTests.js
test_frontend.jsx
.env
.gitignore
eslint.config.js
test_backend.py
JobComparison.jsx
CareerRecommendation.jsx
test_frontend.jsx
frontend > src > frontend.test.jsx > describe("Frontend tests") callback > test("Login page renders
import { MemoryRouter } from "react-router-dom";
import { describe, expect, test } from "vitest";
import Login from "./pages/Login";
describe("Frontend tests", () => {
test("Login page renders correctly", () => {
render(
<MemoryRouter>
<Login />
</MemoryRouter>
);
Problems
Output
Debug Console
Terminal
Ports
toluwaniaderibigbe@Toluwanis-MacBook-Air CareerPilotAI % cd frontend
toluwaniaderibigbe@Toluwanis-MacBook-Air frontend % npm test
> frontend@0.0.0 test
> vitest
DEV v4.1.11 /Users/toluwaniaderibigbe/CareerPilotAI/frontend
src/frontend.test.jsx (1 test) 78ms
Frontend tests (1)
Login page renders correctly 76ms
Test Files 1 passed (1)
Tests 1 passed (1)
Start at 22:27:30
Duration 693ms (transform 25ms, setup 60ms, import 55ms, tests 78ms, environment 416ms)
PASS Waiting for file changes...
press h to show help, press q to quit

6.6 Google Gemini AI Feature Testing

The Google Gemini AI features was tested to ensure that CareerPilot AI could generate interview questions and cover letters based on the users cv and their job description and also needed information filled by the user. The reason for testing is to ensure that requests could be sent from the React frontend to the FastAPI backend, processed through the external AI service and returned to the user interface. The interview feature was tested to confirm that suitable practice questions were generated from Gemini Ai, while the cover-letter feature was tested using the details entered by the user. The testing ensures that the requests were sent through the FastAPI backend to the Gemini API and that was successful the responses were returned and displayed to the user. and also that both AI features works the right way.

6.7 System Evaluation

The evaluation of CareerPilot AI proves that the system achieved its main purpose of helping student and graduates get employed with its career analysis. The application allows users to upload a CV, identify matched and missing skills, it provides an alternative career recommendations and learning roadmaps so the user can learn about there missing skills based on the analysis. other features such as interview practice and cover-letter generation provide extra support when the user is trying to get prepared for their job. The application testing confirmed that the React frontend, FastAPI backend, SQLite database, ESCO data and Google Gemini API were able to work together as one integrated application. the project demonstrates how CV analysis, occupational data and Artificial Intelligence can be combined to create a career support application.

6.8 Limitations and Future Improvements

Although CareerPilot AI achieved its main objectives, but there are still several limitations identified during testing. The CV analysis is based on the user cv and the users job descriptions you need to type your needed skills so the application to know your skills meaning that unclear or incomplete information may affect the skills identified by the system. The Career recommendations depends on the occupation and skills information available from ESCO so it may not cover every possible career path. The interview-practice and cover-letter features rely on the Google Gemini API, so their availability depends on an external service and the generated content may change between requests. Future improvements could be more advanced CV parsing, and improved recommendation methods. The application could also be deployed as an online service, allowing users to access CareerPilot AI without running the frontend and backend locally.

6.9 Summary of Testing and Evaluation

Testing and evaluation were carried out to ensure that all the features of CareerPilot AI functions are working together as an integrated system. The testing covered the backend and APIs, CV processing and skills analysis, ESCO-based career recommendations, frontend integration and Google Gemini AI features. The results proves that the application could process user information, provide career analysis and return the right results to the user interface. testing also helped to identify limitations and areas that should be improved in future development. CareerPilot AI achieved its objectives and it helps users in career planning.

Chapter 7: Conclusions and Future Work

7.1 Project Summary

The aim of the project was to design and develop CareerPilot AI, a web-based career supporting application that allows users understand their existing skills and it also provide an alternative career paths. The system allows users to upload a CV, enter their job description, identify matched and missing skills, and also provide alternative career recommendations, and learning roadmaps for the user to study for their missing skills. It also uses AI for interview practice questions and generate cover letter to help users get prepared for their career. CareerPilot AI is developed as a full-stack application using React with Vite for the frontend, and Python with FastAPI for the backend and SQLite for persistent data storage. ESCO occupation and skills data was integrated for skills analysis and career recommendations, while the Google Gemini API is used for the Artificial Intelligence features. the project achieved its main objective by creating a responsive application that combines CV processing, skills analysis, career recommendations and Artificial Intelligence inside the application. The project demonstrate the practical integration of frontend, backend, database and external API technologies to provide users with career guidance.

7.2 Achievement of Project Objectives

The main objectives of CareerPilot AI was achieved through the development and integration of the system features. a full-stack web application was created using React with Vite for the frontend and Python with FastAPI for the backend. The system allows users to register and log in, upload a CV and receive an analysis of their skills. the project achieved its aim of using cv to analyse skills, matching skills, missing skills, career recommendations, and AI features . The CV processing feature extracts text from the uploaded CV and identifies skills. the skills identified are

compared with ESCO occupation and skills data to identify matched and missing skills and career recommendations. the project also integrate Google Gemini to provide interview practice and cover letter generation.

Testing proves that the frontend, backend, database, CV processing, ESCO integration and AI works together in an application.

7.3 Reflection and Future Work

The development of CareerPilot AI provided experience in building and integrating a full-stack web application. the main lesson learned is how different technologies are connected together to make a complete application. developing the React frontend, and Python FastAPI backend, SQLite database, CV processing, ESCO integration and Google Gemini features required more testing to ensure that the information source is Gemini AI.

The project achieved its aim, but there are still areas where CareerPilot AI could be improved. The accuracy of CV skills identification and career recommendations can still be improved. The system could also provide more career recommendations based user's work experience, qualifications, interests and career goals. The AI features could be extended to provide more detailed interview practice and career guidance. improved security and authentication, support for additional CV formats, and a more advanced user interface. These improvements would make CareerPilot AI more suitable for wider real-world use.

7.4 Final Conclusion

In conclusion, CareerPilot AI demonstrates the development of a full-stack career-support web application that is made up of CV analysis, identifies skills, career recommendations and AI features. The system was developed using React with Vite

for the frontend, Python with FastAPI for the backend, SQLite for persistent data storage, ESCO occupation and skills data as the data source, and Google Gemini for AI-supported features. The application allows users to create an account, upload a CV, identify matched and missing skills, provide career recommendations, interview questions and generate cover letters. all this features are connected together to help users with information that can help them get employed.

Chapter 8: References

1. Python Software Foundation. (2026). *Python Documentation*. at: https://docs.python.org/3/

2. FastAPI. (2026). *FastAPI Documentation: Tutorial – User Guide* at: https://fastapi.tiangolo.com/tutorial/

3. React. (2026). *React Documentation*. Available at: https://react.dev/

4. Vite. (2026). *Vite Documentation: Getting Started* at: https://vite.dev/guide/

5. SQLite. (2026). *SQLite Documentation* at: https://www.sqlite.org/docs.html

6. European Commission. (2026). *European Skills, Competences, Qualifications and Occupations (ESCO)* at: https://esco.ec.europa.eu/en

7. European Commission. (2026). *About ESCO*. at: https://esco.ec.europa.eu/en/about-esco

8. Google. (2026). *Gemini API Reference*. Google AI for Developers at: https://ai.google.dev/api